МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
Институт перспективной инженерии
Межинститутская базовая кафедра
Институт информационных технологий и кибербезопасности

Отчёт к лабораторной работе №5

Тема: **«Структурные паттерны: Фасад, Приспособленец»**

Дисциплина: **«Паттерны проектирования»**

Выполнил:

Студент группы ПИЖ-б-о-23-1
направление подготовки:
09.03.04 «Программная
инженерия»

Астраков Борис Александрович

_____

Проверил:

Ассистент МИБК

Потапов Иван Романович

_____

Ставрополь 2026

Цель работы: реализовать решение задачи из предметной области в соответствии с вариантами (пп. 1.4) с использованием паттернов из списка: «Фасад», «Приспособленец».

Ход работы:

Вариант проекта – игра «Собиратель монет» на pygame.

1. Фасад

Где находится: coin_collector/multiplayer.py, класс TwoPlayerMatch.

Задача: предоставить приложению одну точку входа для правил матча: движение игроков, обновление монет, сбор, подсчёт очков и определение победителя.

```
class TwoPlayerMatch:
    """Facade coordinating movement, coin collection, scoring, and the
result."""

    def __init__(self, level: GameLevel) -> None:
        if len(level.players) != 2:
            raise ValueError("A two-player match requires exactly two
players")
        self.level = level
        self.players = PlayerComposite(level.players)
        self.coin_counts = [0, 0]
        self.scores = [0, 0]
        self.finished = False

    def update(
        self,
        player_one: InputFrame,
        player_two: InputFrame,
        delta_time: float,
    ) -> int:
        if self.finished:
            return 0
        self.players.update(
            (player_one, player_two),
            delta_time,
            self.level.bounds,
        )
        for coin in self.level.coins:
            coin.update(delta_time)
        collected = 0
        remaining: list[Collectible] = []
        for coin in self.level.coins:
            collector = next(
                (
                    index
                    for index, player in enumerate(self.players.players)
```

```
                    if player.rect.colliderect(coin.rect)
                ),
                None,
            )
            if collector is None:
                remaining.append(coin)
            else:
                self.coin_counts[collector] += 1
                self.scores[collector] += coin.value
                collected += 1
        self.level.coins = remaining
        self.finished = not remaining
        return collected
```

Как работает: приложение передаёт два кадра управления в update(), а затем использует итоговые coin_counts, scores, finished и result_message. После сбора последней монеты победитель определяется по количеству монет, а бонусная монета при этом остаётся одной монетой, хотя приносит больше очков.

```
collected = match.update(player_one_frame, player_two_frame, delta_time)
if match.finished:
    print(match.result_message)
```

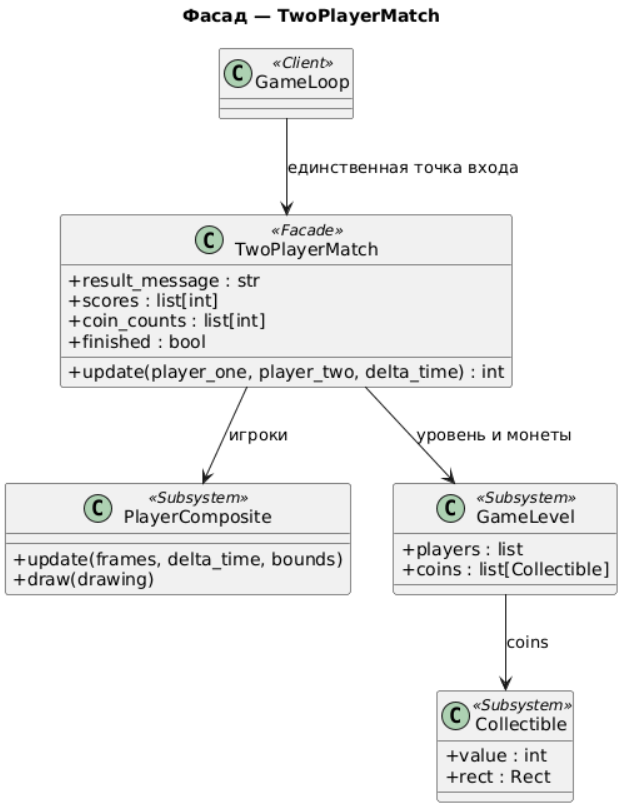


Рисунок 1. Диаграмма паттерна "Фасад"

Таким образом, экран и игровой цикл не содержат подробностей координации двух игроков и правил подсчёта результата.

2. Приспособленец

Где находится: coin_collector/entities.py, неизменяемый CoinStyle; coin_collector/factory.py, CoinStyleFactory.

Задача: не хранить отдельную копию одинаковой палитры в каждой монете.

```
@dataclass(frozen=True)
class CoinStyle:
    shadow: tuple[int, int, int]
    fill: tuple[int, int, int]
    highlight: tuple[int, int, int]
```

```
class CoinStyleFactory:
    """Flyweight factory for immutable visual data shared by coin
instances."""

    _styles: dict[str, CoinStyle] = {"gold": Coin.DEFAULT_STYLE}

    @classmethod
    def get(cls, name: str) -> CoinStyle:
        try:
            return cls._styles[name]
        except KeyError as error:
            raise ValueError(f"Unsupported coin style: {name!r}") from
error
```

Как работает: фабрика стилей выдаёт общий неизменяемый объект палитры "gold". Прототип монеты и все созданные из него экземпляры используют одну ссылку на этот объект; собственные координаты и анимация у монет остаются независимыми.

```
factory = ArcadeObjectFactory()
first = factory.create(ObjectKind.COIN, (120, 150))
second = factory.create(ObjectKind.COIN, (840, 150))
assert first.style is second.style
```

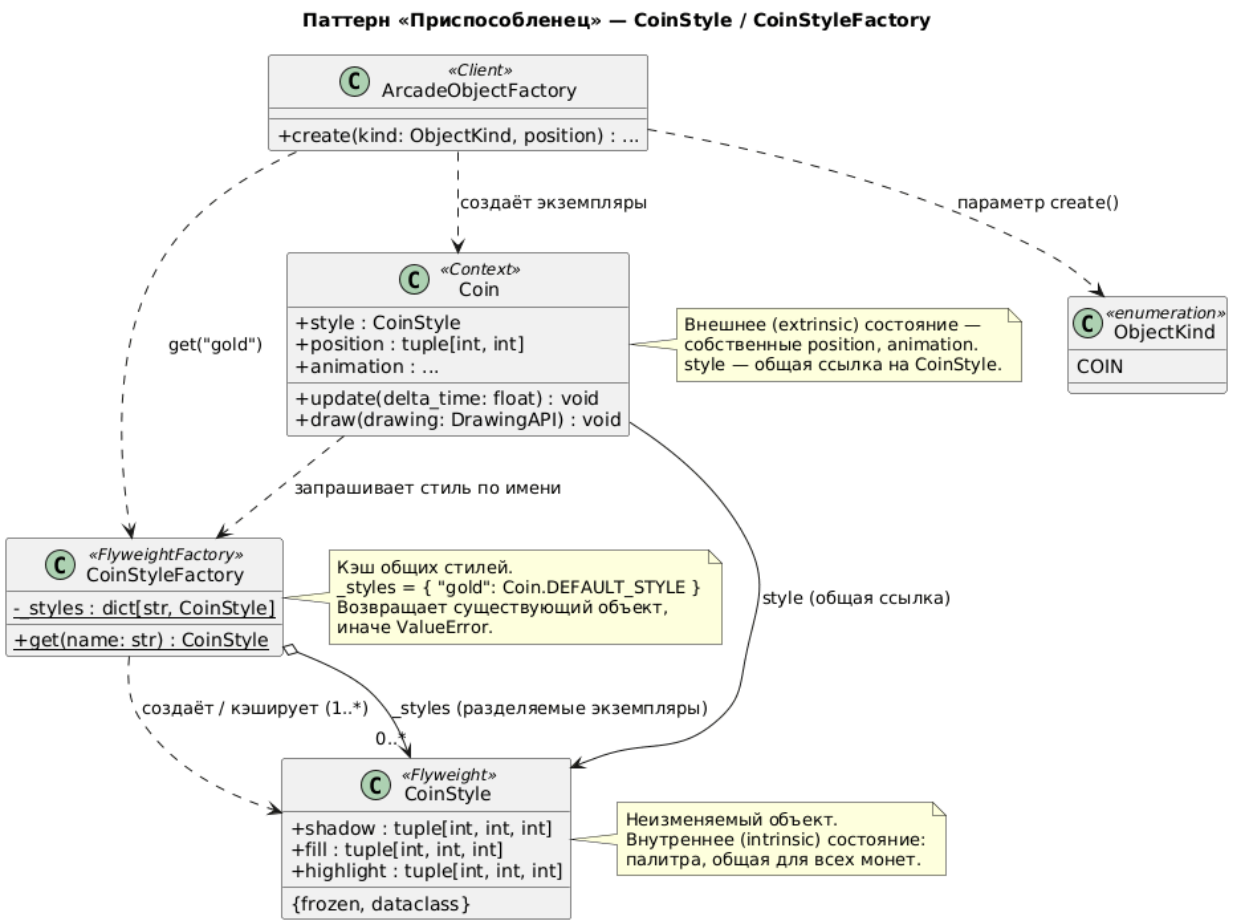


Рисунок 2. Диаграмма паттерна "Приспособленец"

Таким образом, уровень может содержать много визуально одинаковых монет, и их общие данные стиля разделяются безопасно благодаря неизменяемости.

Вывод:

Проделав работу, реализовал решение задачи из предметной области в соответствии с вариантами (пп. 1.4) с использованием паттернов из списка: «Фасад», «Приспособленец».